

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-a968f88f32362a642.jsonl`  
- SHA-256: `da72924fb3112fc4e452b2a1999118ce3bf38983dfc30bd2d2aa23c9de9a2ad1`  
- Source modified: `2026-06-12T10:47:40+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_c5f5bca5-d2a`  
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:45:10.186Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINES.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downsampled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refuted if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [low] Integration checks verified clean (informational) (C:\Users\avidu\Projects\sports-data-collector:n/a)

Lens items checked and found CORRECT, for the record: (1) video_id derivation is an exact mirror of import-local (curate.py:113-114 vs 665-668), including multi-dot filenames (both split('.')[0]) and the lower()/space->underscore transform; the happy-path test asserts 'Match One.mp4' -> 'match_one'. (2) .gitignore's existing data/videos/ rule covers the default proxy dir data/videos/proxies/; the manifest default data/annotation_proxy_manifest.csv is NOT ignored (only data/videos/, data/eval_export*/, data/temp/, data/*.backup-*.db are) ? tracked intentionally and documented 'commit it' in both help text and FAQ. The working tree has an UNCOMMITTED .gitignore edit adding artifacts/ ? unrelated to this branch, don't fold it into this PR. (3) No new pip deps: proxy.py is stdlib-only (csv/hashlib/os/shutil/subprocess/dataclasses/typing); typing uses Optional/List/Dict ? fine for requires-python >=3.9; lazy-import + print-style UX match sibling commands. (4) CI (plain `python -m pytest -q` on ubuntu, no ffmpeg) is safe: all ffmpeg/ffprobe touchpoints are mocked; full suite verified locally: 159 passed in 2.49s, and `prep-annotation-proxy --help` renders without argparse errors; no flag/dest collisions (subparsers are isolated). (5) Downstream: convert-cvat/validate-delivery use --video-path only for fps probing (probe_resolution_fps, avg_frame_rate); export's manifest.json pairs DB-registered local_path files ? proxies are never registered so export/eval manifests are unaffected; PIPELINE_ROLE.md's 'downstream processing must run on THAT file' contract is restated consistently in the module docstring, CLI next-steps, and the new FAQ. (6) ADR-002's ratified knobs match the defaults (1080p not 720, CRF 19 in the 18-20 band, -g 15, audio kept, -fps_mode passthrough, no -r/trim).
Proposed fix: No action needed; listed so the parent review does not re-audit these.

2. user (2026-06-12T10:45:20.973Z)

```
fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-
proxy) (#21)
a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)
docs/INGESTION_PIPELINE.md | 29 ++++
src/cli/curate.py | 147 +++++
src/core/proxy.py | 242 +++++
tests/test_proxy.py | 309 +++++
4 files changed, 724 insertions(+), 3 deletions(-)
```

3. user (2026-06-12T10:45:21.760Z)

```
1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the  $\pm 0.3$  s boundary bar.
7      The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8      downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9      so frame N on the proxy IS frame N on the original and returned labels apply to
10     the original unchanged.
11
12     The ADR-002 proxy contract enforced here:
13     - video timestamps pass through untouched (`-fps_mode passthrough`, never
14       ``-r``): same frame count, same fps, no trim, no leading padding;
15     - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16       failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17       golden labels;
18     - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19       survives the handoff (the indexer keys videos by file MD5; downstream
20       processing must keep running on the ORIGINAL file).
21     """
22     import csv
23     import hashlib
24     import os
25     import shutil
26     import subprocess
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional
29
30
31     class ProxyError(RuntimeError):
32         """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35     #: Columns of the provenance manifest (one row appended per generated proxy).
36     MANIFEST_FIELDS = [
37         "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38         "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39         "encoder_settings", "created_at",
40     ]
41
42     # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43     # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44     # back-stepping; CRF 19 keeps the ball/shuttle legible.
45     DEFAULT_HEIGHT = 1080
46     DEFAULT_CRF = 19
47     DEFAULT_GOP = 15
48     DEFAULT_PRESET = "medium"
49
50
51     @dataclass
```

```

52 class ProbeInfo:
53     """The frame?time identity of one video stream, as ffprobe reports it."""
54     width: Optional[int] = None
55     height: Optional[int] = None
56     avg_fps: Optional[float] = None      # avg_frame_rate (frames / duration)
57     r_fps: Optional[float] = None        # r_frame_rate (the container's base rate)
58     nb_frames: Optional[int] = None      # demuxed packet count == frame count
59     duration: Optional[float] = None     # video stream duration, else container
60
61     @property
62     def resolution(self) -> str:
63         if self.width and self.height:
64             return f"{self.width}x{self.height}"
65         return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None
72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:
77                 return None
78             return num_f / den_f
79         return float(value)
80     except ValueError:
81         return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffprobe (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffprobe must stop it, not degrade it.
91     """
92     if not shutil.which("ffprobe"):
93         raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffprobe", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106         ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffprobe failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:

```

```

117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)
125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's
129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),
137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )
142
143
144     def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145         """True when the stream looks variable-frame-rate (or we cannot tell).
146
147         For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148         `frame / fps` time recompute is approximate ? conservative callers should
149         require an explicit opt-in before proxying such a source.
150         """
151         if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152             return True
153         return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156     def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                             crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                             preset: str = DEFAULT_PRESET) -> List[str]:
159         """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161         Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162         trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163         1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164         never upscales. Audio is kept (contact sound is the annotator's best
165         boundary cue).
166         """
167         return [
168             "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169             "-i", src,
170             "-map", "0:v:0", "-map", "0:a?",
171             "-vf", f"scale=-2:min({height}\\,ih)",
172             "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173             "-g", str(gop), "-pix_fmt", "yuv420p",
174             "-fps_mode", "passthrough",
175             "-c:a", "aac", "-b:a", "160k",
176             "-movflags", "+faststart",
177             dst,
178         ]
179
180
181     def run_ffmpeg(cmd: List[str]) -> None:

```

```

182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
194     """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []
202
203     if src.nb_frames is None or proxy.nb_frames is None:
204         errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
205     elif src.nb_frames != proxy.nb_frames:
206         errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208     if src.avg_fps is None or proxy.avg_fps is None:
209         errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210     elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211         errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213     if src.duration is None or proxy.duration is None:
214         errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215     else:
216         tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217         if abs(proxy.duration - src.duration) > tol:
218             errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                           f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221     return errors
222
223
224 def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225     """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226     digest = hashlib.md5()
227     with open(path, "rb") as f:
228         for chunk in iter(lambda: f.read(chunk_size), b""):
229             digest.update(chunk)
230     return digest.hexdigest()
231
232
233 def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234     """Append one provenance row, writing the header only on first creation."""
235     parent = os.path.dirname(os.path.abspath(manifest_path))
236     os.makedirs(parent, exist_ok=True)
237     is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238     with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239         writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240         if is_new:
241             writer.writeheader()
242         writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})

```

4. assistant (2026-06-12T10:45:29.078Z)

Now let me look at the `curate.py` diff, the `.gitignore`, and the ADR.

5. user (2026-06-12T10:45:29.783Z)

```
diff --git a/src/cli/curate.py b/src/cli/curate.py
index fde4826..a6d76c2 100644
--- a/src/cli/curate.py
+++ b/src/cli/curate.py
@@ -643,6 +643,120 @@ class SportsDataCLI:
     print(f"Skipped (no annotations) : {skipped_empty}")
     print(f"Manifest          : {manifest_path}\n")

+
+ def prep_annotation_proxy(self, args):
+     """Re-encode a video into a frame-identical annotation proxy (ADR-002).
+
+     The proxy is what the annotation vendor receives: downscaled and
+     scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
+     but with the source's EXACT frame count / fps / timeline, so frame N
+     on the proxy is frame N on the original and returned labels apply to
+     the original unchanged. Parity is verified after the encode; a proxy
+     that fails verification is deleted, never shared. Provenance (source +
+     proxy MD5s) is appended to the proxy manifest ? downstream processing
+     (import-local, export, the indexer) keeps running on the ORIGINAL file.
+     """
+     from src.core import proxy as proxylib
+
+     src = args.video_path
+     if not os.path.exists(src):
+         print(f"Error: video '{src}' not found.")
+         sys.exit(1)
+
+     stem = os.path.basename(src).split('.')[0]
+     # Mirror import-local's video_id derivation so the manifest row and a
+     # later DB registration of the same source agree on the key.
+     video_id = args.video_id or stem.lower().replace(" ", "_")
+     out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
+     if os.path.abspath(out) == os.path.abspath(src):
+         print("Error: --out must differ from --video-path.")
+         sys.exit(1)
+     if os.path.exists(out) and not args.force:
+         print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
+         sys.exit(1)
+
+     try:
+         src_info = proxylib.probe_video(src)
+     except proxylib.ProxyError as e:
+         print(f"Error: {e}")
+         sys.exit(1)
+     if src_info.avg_fps is None:
+         print(f"Error: could not determine fps of '{src}' ? the frame->time "
+               f"contract needs it.")
+         sys.exit(1)
+     if proxylib.is_vfr(src_info) and not args.allow_vfr:
+         print(f"Error: '{src}' looks variable-frame-rate "
+               f"r={src_info.r_fps} vs avg={src_info.avg_fps}.")
+         print(" The vendor pipeline recomputes times as frame/fps, which is only "
+               "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
+               "proceed anyway (timestamps stay identical to the original, but "
+               "frame/fps seconds are approximate for BOTH files).")
+         sys.exit(1)
```

```

+ os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
+ cmd = proxylib.build_proxy_command(src, out, height=args.height,
+                                   crf=args.crf, gop=args.gop,
+                                   preset=args.preset)
+ print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
+       f"f"fps {src_info.avg_fps:g} preserved)...")
+ try:
+     proxylib.run_ffmpeg(cmd)
+     proxy_info = proxylib.probe_video(out)
+ except proxylib.ProxyError as e:
+     print(f"Error: {e}")
+     sys.exit(1)
+
+ errors = proxylib.verify_parity(src_info, proxy_info)
+ if errors:
+     print("Error: proxy FAILED frame<->time parity verification:")
+     for err in errors:
+         print(f" - {err}")
+     try:
+         os.remove(out)
+         print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
+               f"f"poisons golden labels).")
+     except OSError:
+         print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
+               f"f"share it with the vendor.")
+     sys.exit(1)
+
+ print("Hashing source + proxy (provenance)...")
+ source_md5 = proxylib.md5_file(src)
+ proxy_md5 = proxylib.md5_file(out)
+
+ from datetime import datetime, timezone
+ proxylib.append_manifest_row(args.manifest, {
+     "video_id": video_id,
+     "source_path": os.path.abspath(src),
+     "source_md5": source_md5,
+     "proxy_path": os.path.abspath(out),
+     "proxy_md5": proxy_md5,
+     "fps": f"{src_info.avg_fps:g}",
+     "frame_count": src_info.nb_frames,
+     "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None else "",
+     "source_resolution": src_info.resolution,
+     "proxy_resolution": proxy_info.resolution,
+     "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
+                          f"preset={args.preset} height<={args.height} "
+                          f"f"fps_mode=passthrough audio=aac160k"),
+     "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
+ })
+
+ src_mb = os.path.getsize(src) / (1024 * 1024)
+ out_mb = os.path.getsize(out) / (1024 * 1024)
+ print("\n=== Annotation proxy ready ===")
+ print(f" Video ID      : {video_id}")
+ print(f" Source        : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
+ {source_md5})")
+ print(f" Proxy         : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
+ {proxy_md5})")
+ print(f" Parity        : VERIFIED ? {src_info.nb_frames} frames @ {src_info.avg_fps:g}
+ fps on both")
+ print(f" Manifest      : {args.manifest}")
+ print("\nNext:")
+ print(" 1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
+       "box of every rally on the exact contact/dead frame).")
+ print(" 2. On delivery, convert-cvat may use --video-path with either file "
+       "(identical fps/frame count by construction).")

```

```

+         print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
+               "indexer keys videos by its MD5.\n")
+
+     def convert_cvat(self, args):
+         """Convert a vendor CVAT rally export into our canonical per-rally CSV.

@@ -1017,6 +1131,37 @@ def main():
    fetch_parser.add_argument("--retry-backoff", type=float, default=20.0, metavar="SECONDS",
                              help="Base backoff between retries; grows linearly per attempt
(default: 20)")

+    # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
+    prep_parser = subparsers.add_parser(
+        "prep-annotation-proxy",
+        help="Re-encode a video into a light, frame-identical annotation proxy for "
+        "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "
+        "count/fps/timeline, parity-verified, provenance recorded")
+    prep_parser.add_argument("--video-path", required=True,
+                             help="The ORIGINAL source video (stays the system-of-record
file)")
+    prep_parser.add_argument("--out",
+                             help="Proxy output path (default:
data/videos/proxies/<stem>_proxy.mp4)")
+    prep_parser.add_argument("--video-id",
+                             help="Manifest key (default: derived from the filename, like
import-local)")
+    prep_parser.add_argument("--height", type=int, default=1080,
+                             help="Max proxy height; never upscales (default: 1080 ? "
+                             "720 risks losing the table-tennis ball)")
+    prep_parser.add_argument("--crf", type=int, default=19,
+                             help="x264 quality, lower=better (default: 19)")
+    prep_parser.add_argument("--gop", type=int, default=15,
+                             help="Max keyframe interval in frames; small = snappy "
+                             "frame-stepping (default: 15)")
+    prep_parser.add_argument("--preset", default="medium",
+                             help="x264 speed/size preset (default: medium)")
+    prep_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
+                             help="Provenance CSV to append to (default:
data/annotation_proxy_manifest.csv; "
+                             "commit it ? it is the proxy<->original mapping of record)")
+    prep_parser.add_argument("--allow-vfr", action="store_true",
+                             help="Proceed on a variable-frame-rate source (frame/fps "
+                             "seconds are approximate; default: refuse)")
+    prep_parser.add_argument("--force", action="store_true",
+                             help="Overwrite an existing proxy file")
+
+    # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
+    cvat_parser = subparsers.add_parser(
+        "convert-cvat",

@@ -1077,6 +1222,8 @@ def main():
    cli.curate()
    elif args.command == "import-local":
        cli.import_local(args)
+    elif args.command == "prep-annotation-proxy":
+        cli.prep_annotation_proxy(args)
+    elif args.command == "convert-cvat":
+        cli.convert_cvat(args)
    elif args.command == "validate-delivery":

```

6. user (2026-06-12T10:45:30.267Z)

```

1      # ---- Python ----
2      __pycache__/
3      *.py[cod]

```

```

4      *$py.class
5      *.egg-info/
6      .eggs/
7      build/
8      dist/
9      .installed.cfg
10     *.egg
11
12     # ---- Test / coverage ----
13     .pytest_cache/
14     .coverage
15     .coverage.*
16     htmlcov/
17     .tox/
18     .cache/
19
20     # ---- Virtual environments ----
21     .venv/
22     venv/
23     env/
24     ENV/
25
26     # ---- Secrets / local config ----
27     .env
28     .env.*
29
30     # ---- OS / editor cruft ----
31     .DS_Store
32     Thumbs.db
33     .idea/
34     .vscode/
35     *.swp
36
37     # ---- Project: large media kept outside git (see README) ----
38     # The video corpus is hundreds of MB+; only metadata (the SQLite DB) and the
39     # small source annotation CSVs are version-controlled. Re-fetch videos from the
40     # stored video_url / local_path per the manifest.
41     data/videos/
42
43     # ---- Generated eval/training exports (regenerated via `curate export`) ----
44     data/eval_export*/
45
46     # ---- Generated temp artifacts (crawler scratch space; regenerated each run) ----
47     data/temp/
48
49     # ---- Per-video observability reports (written next to videos / inputs at runtime) ----
50     *.report.json
51     *.report.md
52     *.summary.md
53
54     # Local DB backups (e.g. data/sports_metadata.backup-YYYY-MM-DD.db) ? never commit
55     data/*.backup-*.db
56
57     # Model artifact bundles: weights are NEVER committed (private Releases); the
58     # reviewable surface is each bundle's manifest.json (ADR-008).
59     artifacts/
60

```

7. user (2026-06-12T10:45:43.647Z)

```

M .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git
touches it
diff --git a/.gitignore b/.gitignore
index 52dac92..ac1420f 100644

```

```

--- a/.gitignore
+++ b/.gitignore
@@ -53,3 +53,7 @@ data/temp/

# Local DB backups (e.g. data/sports_metadata.backup-YYYY-MM-DD.db) ? never commit
data/*.backup-*.db
+
+# Model artifact bundles: weights are NEVER committed (private Releases); the
+# reviewable surface is each bundle's manifest.json (ADR-008).
+artifacts/
e4dd8f1

```

8. user (2026-06-12T10:45:44.270Z)

```

    if sport_str not in self.config.get("supported_sports", ["badminton"]):
        print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
        sys.exit(1)

    # Validate video path
    if not os.path.exists(args.video_path):
        print(f"Warning: Local video path '{args.video_path}' does not exist on disk, but
continuing registration.")

    # Validate annotation file
    if not os.path.exists(args.annotations_path):
        print(f"Error: Annotations file '{args.annotations_path}' not found.")
        sys.exit(1)

    match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
    video_id = args.video_id or match_name.lower().replace(" ", "_")

    # Parse license
    try:
        license_type = LicenseType(args.license)
    except ValueError:
        print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown (non-
commercial).")
        license_type = LicenseType.UNKNOWN

    is_commercial = self.validator.check_license_commercial(license_type)

    # Optional: normalize a raw CSV annotator export before parsing (drops unusable
    # rows, renumbers fractional/duplicate rally ids) so the landmines below can't fire.
    annotations_path = args.annotations_path
    if getattr(args, "normalize", False) and annotations_path.lower().endswith(".csv"):
        from src.cli.normalize_annotations import normalize_file
        normalized_path = annotations_path + ".normalized.csv"

```

9. user (2026-06-12T10:45:58.671Z)

```

28:def test_build_command_preserves_the_timeline():
44:def test_build_command_custom_knobs():
52:def test_parse_rate():
62:def test_is_vfr():
77:def test_verify_parity_ok_and_tolerates_audio_priming():
84:def test_verify_parity_catches_frame_count_change():
89:def test_verify_parity_catches_fps_change():
94:def test_verify_parity_catches_trim():
99:def test_verify_parity_loud_on_unknown_identity():
104:def test_md5_file(tmp_path):
110:def test_append_manifest_header_written_once(tmp_path):
137:def test_probe_video_parsing_ffprobe_json(tmp_path):
151:def test_probe_video_requires_ffprobe(tmp_path):
159:def test_probe_video_missing_file():

```

```

165: def test_probe_video_no_video_stream(tmp_path):
193:     video_path = str(tmp_path / "Match One.mp4"),
207: def test_prep_proxy_happy_path(cli, tmp_path):
209:     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
226:     assert row["video_id"] == "match_one"
227:     assert row["source_md5"] == md5_file(str(tmp_path / "Match One.mp4"))
236: def test_prep_proxy_parity_failure_deletes_proxy(cli, tmp_path):
238:     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
255: def test_prep_proxy_refuses_vfr_without_flag(cli, tmp_path):
257:     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
267: def test_prep_proxy_allow_vfr_proceeds(cli, tmp_path):
269:     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
285: def test_prep_proxy_refuses_existing_output(cli, tmp_path):
287:     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
296: def test_prep_proxy_missing_source(cli, tmp_path):
303: def test_prep_proxy_out_equals_source_refused(cli, tmp_path):
304:     src = tmp_path / "Match One.mp4"

```

10. user (2026-06-12T10:45:59.817Z)

Exit code 2

ci.yml

name: CI

on:

push:

branches: [master]

pull_request:

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-python@v5

with:

python-version: "3.11"

- name: Install dependencies

run: |

python -m pip install --upgrade pip

pip install -e ".[dev]"

- name: Run tests

run: python -m pytest -q

pyproject.toml:10:requires-python = ">=3.9"

11. user (2026-06-12T10:46:11.403Z)

4: Everything runs offline: ffmpeg/ffprobe are mocked; only the pure logic

11: from unittest import mock

122: # probe_video (ffprobe mocked)

#

140: with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"), \

141: mock.patch("src.core.proxy.subprocess.run",

154: with mock.patch("src.core.proxy.shutil.which", return_value=None):

160: with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"):

168: with mock.patch("src.core.proxy.shutil.which", return_value="ffprobe"), \

169: mock.patch("src.core.proxy.subprocess.run",

216: with mock.patch("src.core.proxy.probe_video", side_effect=probes), \

217: mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):

246: with mock.patch("src.core.proxy.probe_video", side_effect=probes), \

247: mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):

260: with mock.patch("src.core.proxy.probe_video", return_value=vfr_info):

279: with mock.patch("src.core.proxy.probe_video", side_effect=[vfr, vfr_proxy]), \

280: mock.patch("src.core.proxy.run_ffmpeg", side_effect=fake_ffmpeg):

12. user (2026-06-12T10:46:12.426Z)

```
20:from src.core.downloader import download_best_video, probe_resolution_fps, height_to_label
104:    # Validate video path
105:    if not os.path.exists(args.video_path):
106:        print(f"Warning: Local video path '{args.video_path}' does not exist on disk,
but continuing registration.")
113:    match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
173:    local_path=args.video_path,
458:    _, height, _ = probe_resolution_fps(dest_path)
660:    src = args.video_path
671:    print("Error: --out must differ from --video-path.")
755:    print(" 2. On delivery, convert-cvat may use --video-path with either file "
776:          video_path=args.video_path, step=args.step)
795:    records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
811:    f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
816:    Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
834:    result = convert(args.input, fps=args.fps, video_path=args.video_path)
878:    video_path=args.video_path, video_duration_s=duration_s,
1090:    import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
1140:    prep_parser.add_argument("--video-path", required=True,
1173:    cvat_parser.add_argument("--video-path",
1186:    vd_parser.add_argument("--input", help="CVAT XML export (use with --video-path/--fps)")
1188:    vd_parser.add_argument("--video-path",
```

13. user (2026-06-12T10:46:19.815Z)

```
369:    video_path: Optional[str] = None,
374:    ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
380:    if fps is None and video_path:
381:        _, _, probed = probe_resolution_fps(video_path)
385:        "Could not determine fps. Pass fps=... or a readable video_path so "
76:## ADR-002 ? Annotation proxies are generated in-repo (`curate prep-annotation-proxy`), not
by a one-off tool
```

14. user (2026-06-12T10:46:20.468Z)

feat/prep-annotation-proxy

15. user (2026-06-12T10:46:29.581Z)

ADR-002 ? Annotation proxies are generated in-repo (`curate prep-annotation-proxy`), not by a one-off tool

Status: Accepted · **Date:** 2026-06-12 (owner-ratified)

Context

The Roora pilot showed the vendor annotates in CVAT directly on our **4K / 59.94 fps originals** and copes with the decode cost by sampling **every 30th frame**. The result (pickleball + table-tennis review, 2026-06-10): **every rally boundary off by ~0.5 s**, up to **~2 s**, against the **±0.3 s** bar (Brief §4) ? all 24 boundaries sit exactly on the 30-frame grid (~0.5 s at 59.94 fps), so they **structurally cannot** meet spec. The same grid was flagged **[HIGH]** in the earlier badminton round (`data/roora\_badminton\_issues.txt`); it is their standing workflow, not a one-off lapse.

Phase B makes this untenable, not just suboptimal: **~12?15 videos per sport** at **~30 min** each (Brief §3). A 30-minute 4K/59.94 file is **>100k frames**; frame-accurate annotation (frame-step 1) on 4K originals at that scale is not realistic for the vendor. A lightweight **annotation proxy** (downscaled, scrub-friendly re-encode of the identical timeline) is therefore close to a **precondition for Phase B**, not an optimization.

The pipeline is proxy-friendly by construction: annotations join by the human-assigned

`video\_id` (not file MD5), and `convert-cvat` recomputes all times as `frame / fps`, ignoring vendor time attributes. The Brief's "do not re-encode" rule binds the *vendor* (timeline + checksum identity of the file they annotate), not us ? provided downstream processing always runs on the **original** file (see `PIPELINE\_ROLE.md`: the indexer keys by MD5).

Decision

****We pre-process every video before vendor handoff into an annotation proxy, and the generator lives in this repo as a `curate prep-annotation-proxy` subcommand**** ? not a standalone script, and not in the indexer.

Proxy contract (invariants the tool must enforce)

- ****Same fps as the source ? never reduce it.**** At a lower fps every CVAT frame index changes meaning, and the vendor's 30-frame sampling habit at 30 fps would become a ****1.0 s**** grid ? twice as coarse. Same frame count, no trimming, no leading padding ? proxy seconds map 1:1 onto the original.
- ****Downscale 4K ? 1080p**** (not 720p: table-tennis ball visibility), CRF ~18?20, ****dense keyframes**** (`-g 15`) so frame-stepping/back-scrubbing is fast, ****audio kept**** (contact sound is the annotator's best boundary cue).
- ****Verification built in:**** assert proxy frame count == original frame count and durations match (also surfaces VFR sources, where `frame / fps` is already unsafe).
- ****Provenance:**** emit/append the intake-manifest row ? `video\_id`, fps, frame count, ****original MD5 + proxy MD5**** ? so the proxy?original mapping is durable and downstream processing provably targets the original.

Rationale (why the collector, why not a loose script)

- ****This repo owns the vendor flow end-to-end**** (`docs/annotation/`, intake manifest, `convert-cvat` ? `validate-delivery` ? `import-local`); proxy prep is simply command #0 of that existing pipeline, and it already shells out to ffprobe.
- ****The ffmpeg invocation is the trivial 20%**** The durable value is the parity verification + manifest provenance ? exactly what one-off scripts skip. We have precedent for the failure mode: unfilled intake-manifest TODOs, eval artifacts lost in `%TEMP%`, and indexer predictions once keyed under a *downscaled copy's* MD5.
- ****Phase-B frequency:**** ~40 runs across sessions/operators ? needs tests + CI, i.e. repo-resident code.
- ****Rejected ? standalone script:**** acceptable stopgap for an immediate redelivery, but no durable verification/provenance. ****Rejected ? indexer:**** it has the re-encode primitive (`extract\_video\_chunk`) but is the product pipeline; the vendor handoff workflow, manifest, and ingestion all live here.

Consequences / follow-ups

- ****A proxy alone does not fix boundaries.**** It removes the performance excuse; the binding fix is the convention from the 2026-06-10 review, to be written into the Phase-B brief revision: *annotate at frame step 1; place the first/last box of every rally on the exact contact/dead frame; we supply annotation-optimized proxies.*

16. user (2026-06-12T10:46:30.823Z)

```
544:         print("\nApplied. Re-run `curate export` to regenerate the manifest without
these videos.")
545:     else:
546:         print("\nDry-run only. Re-run with --apply to persist, then `curate export`.")
547:
548:     def export_eval_set(self, args):
549:         --
553:         `out_dir>/manifest.json` that pairs each CSV with its video file. The
554:         manifest is the contract the downstream eval/training driver reads: the
555:         indexer keys videos by *file MD5*, not by our `video_id`, so it must run
556:         against the exact `local_path`/`video_url` recorded here.
```

17. assistant (2026-06-12T10:46:38.550Z)

All documentary claims check out. Last verification: run the suite and the help render.

18. user (2026-06-12T10:46:42.689Z)

```
..... [ 90%]  
..... [100%]  
159 passed in 2.49s
```

19. user (2026-06-12T10:46:48.662Z)

```
usage: curate.py prep-annotation-proxy [-h] --video-path VIDEO_PATH  
                                         [--out OUT] [--video-id VIDEO_ID]  
                                         [--height HEIGHT] [--crf CRF]  
                                         [--gop GOP] [--preset PRESET]  
                                         [--manifest MANIFEST] [--allow-vfr]  
                                         [--force]
```

options:

```
-h, --help            show this help message and exit  
--video-path VIDEO_PATH  
                        The ORIGINAL source video (stays the system-of-record  
                        file)  
--out OUT              Proxy output path (default:  
                        data/videos/proxies/<stem>_proxy.mp4)  
--video-id VIDEO_ID    Manifest key (default: derived from the filename, like
```

22 tests collected in 0.26s

20. user (2026-06-12T10:46:55.873Z)

```
4:+++ b/docs/INGESTION_PIPELINE.md  
10:### The five commands (in order)  
13:## 0. BEFORE handoff: re-encode the source into a frame-identical annotation proxy  
14:## (ADR-002 ? downscaled + scrub-friendly so the vendor can work at frame step 1;  
15:## same frame count/fps/timeline, parity-verified, provenance recorded).  
16:## Share the PROXY with the vendor; everything downstream keeps using the ORIGINAL.  
17:python -m src.cli.curate prep-annotation-proxy --video-path match.mp4  
18:~  
28:~It depends on the consumer. For highlight compilation, ~0.5s sampling slack is  
29:~absorbed by the compiler's clip padding, so `validate-delivery` doesn't flag sampling  
30:~density. For golden training/eval labels the bar is ±0.3s (Brief §4), and the  
31:~pickleball/table-tennis pilot showed a 30-frame sampling grid at 59.94 fps puts *every*  
32:~boundary ?0.5s off ? structurally out of spec. That's why videos are handed off as  
33:~annotation *proxies* (step 0): light enough to annotate at frame step 1, with the  
34:~first/last box of each rally on the exact contact/dead frame.  
35:~  
36:### What's an annotation proxy, and why not annotate the original?  
37:~A 4K/59.94 original is too heavy to frame-step in CVAT ? at Phase-B scale (~30-minute  
38:~videos, >100k frames each) it's not realistic at all, which is what pushed the pilot  
39:~vendor onto the coarse sampling grid above. `prep-annotation-proxy` re-encodes the  
40:~source downscaled (?1080p) with dense keyframes for snappy scrubbing while preserving  
41:~the *exact frame count, fps, and timeline* (verified after the encode; a failed proxy  
42:~is deleted). Frame N on the proxy IS frame N on the original, so returned labels apply  
43:~to the original unchanged. Provenance (source + proxy MD5) is appended to  
44:~`data/annotation_proxy_manifest.csv` ? commit it. The proxy is ONLY for annotation:  
45:~`import-local`, `export`, and the indexer (which keys videos by file MD5) keep using  
46:~the original. Full rationale: ADR-002 in `docs/DECISIONS.md`.  
44:~exact `local_path`, and any downstream processing must run on *that* file. This is
```

21. user (2026-06-12T10:47:35.663Z)

Structured output provided successfully

22. assistant (2026-06-12T10:47:40.141Z)

Verification complete. The informational claim's assertions were all confirmed accurate; no

defect exists and no action is needed.